

Scalable Validation and Optimized Simulation for Quantum Machine Learning

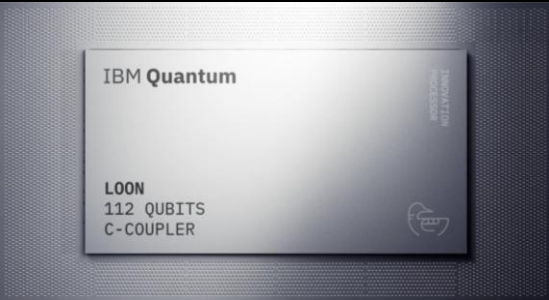
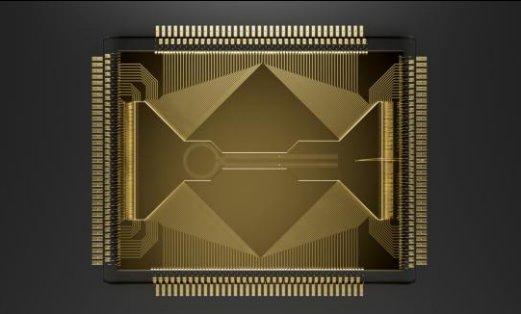
Tutorials · QCE 26

Yun Yuan (Pika) Wang, Sr. Quantum Solutions Architect | NVIDIA
Kuan-Cheng (Louis) Chen, Lead Quantum Researcher | Imperial College London / Jij Inc.
Tai-Yue Li, Senior Researcher | NCHC

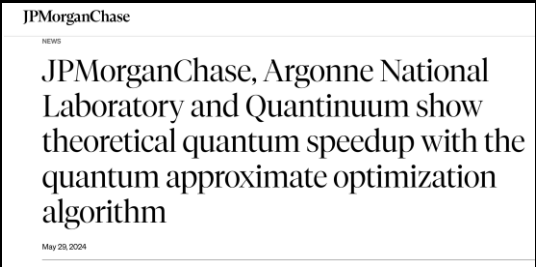
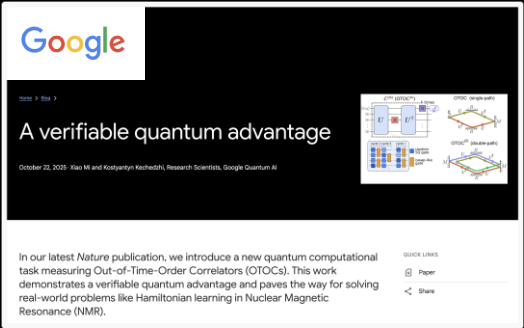
Jiun-Cheng Jiang, Solution Architect | NVIDIA
Kevin Ho, Senior Researcher | NCHC

2025 - Year of Quantum Breakthroughs

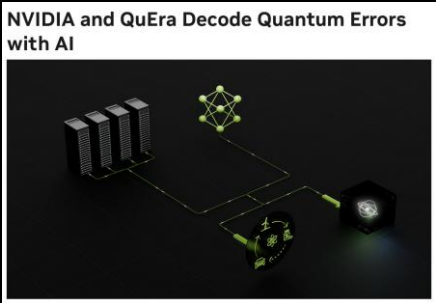
Better Quantum Hardware



Better Quantum Applications



Scaling QC with AI





NVIDIA is not building
Qubits



NVIDIA is building all
Accelerated Quantum
Supercomputers

AI and Quantum

AI to Advance Quantum Computing



Algorithms

QEC

Calibration

Digital Twins

Quantum Computing to Advance AI



Hybrid Applications

Quantum Data
Generation

Expand Frontier of
Accelerated Computing

The Generative Quantum Eigensolver

First demonstration of GPT-generated circuits



Challenge

- Variational quantum algorithms offered promise for running drug-discovery applications on small quantum devices - but **suffer from serious optimization issues**.
- Many of these problems are tied to how circuits are parametrized.

Solution

- The generative quantum eigensolver acts like a Large Language Model – but generating quantum circuits from quantum operations, rather than sentences from words.
- Using a generative model like GPT to create quantum circuits **avoids the limitations of traditional variational quantum algorithms**

$\vec{j} \sim$ "Once upon a time . . . happily ever after"
word_{j₁} word_{j₂} word_{j₃} word_{j₄} . . . word_{j_{N-2}} word_{j_{N-1}} word_{j_N} **LLM**

$\vec{j} \sim$
A diagram of a quantum circuit. It consists of multiple horizontal lines representing qubits. The circuit is divided into sections by vertical lines. Each section contains a set of colored squares (black, green, blue, yellow) representing quantum gates. The sections are labeled U_{j₁}, U_{j₂}, U_{j₃}, ..., U_{j_N}. **GPT-QE**



New approach in
Using AI for building
quantum applications



Can be
extended to various
application
areas

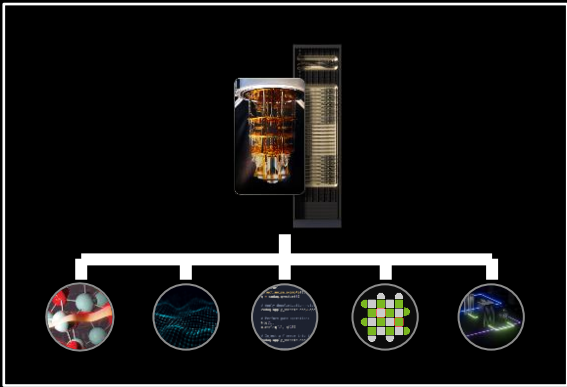
40X

Speedup over
CPU when
running GQE on
GPU





Running useful quantum applications and hardware requires **AI supercomputing**



Developing useful quantum applications and hardware requires **AI supercomputing**

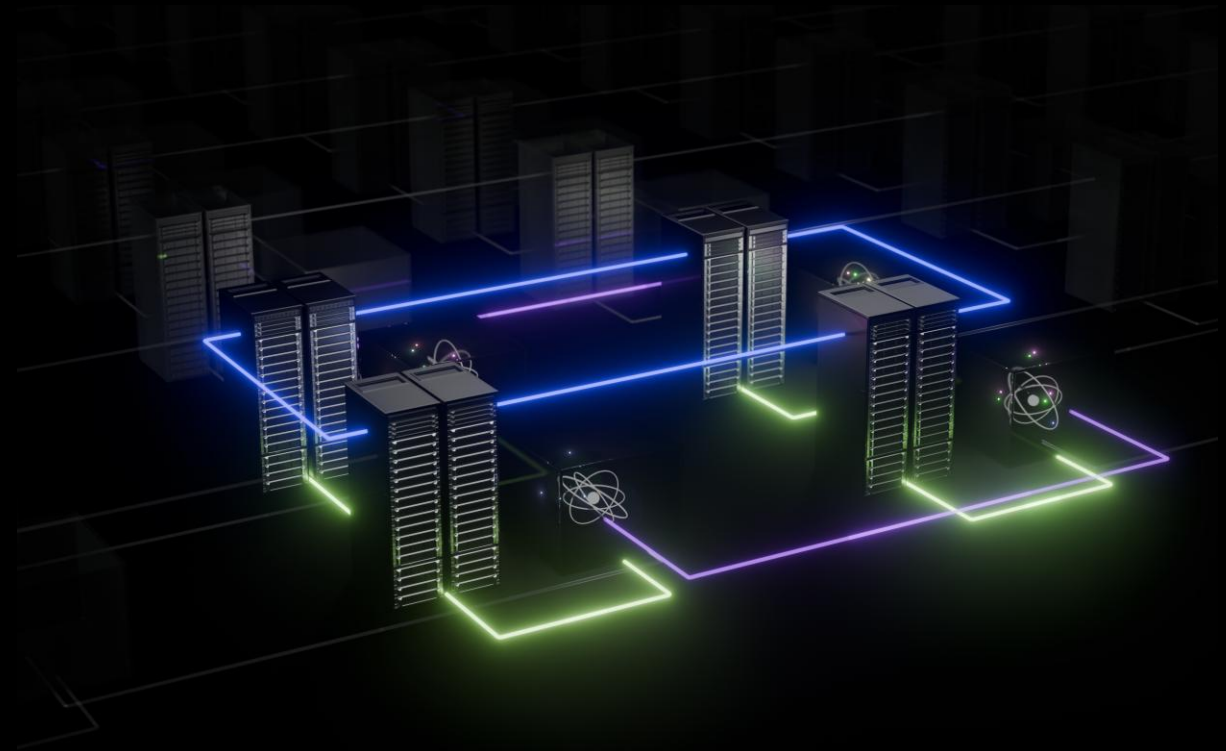
Quantum-GPU Supercomputing

All valuable quantum computing will be **hybrid**

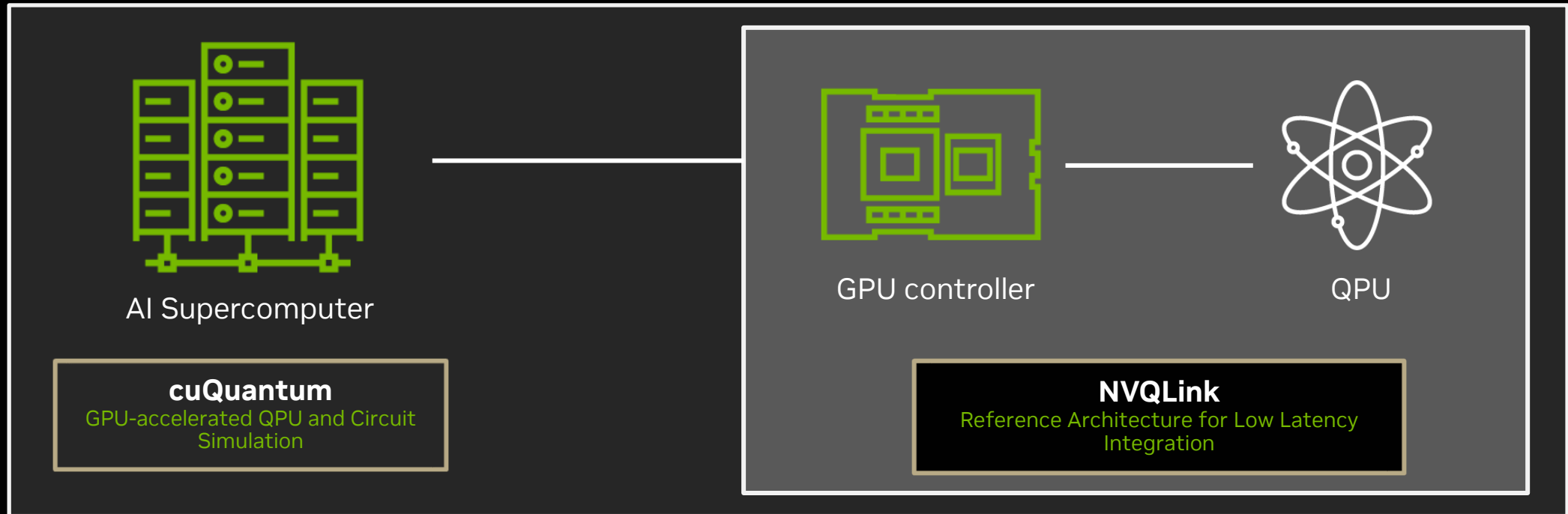
GPUs and AI will prove essential

Petascale compute needed for useful Fault Tolerant Quantum Computers

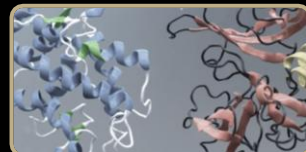
Developer ecosystem is critical



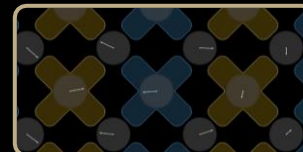
NVIDIA Open Quantum Platform



CUDA-Q
Quantum-GPU Platform

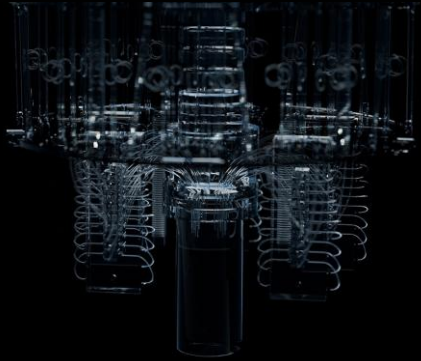


CUDA-Q Solvers

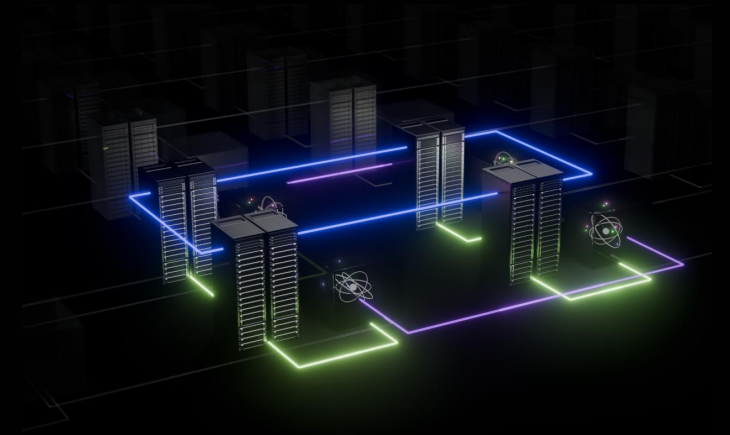


CUDA-Q QEC

The Path from Qubits to Supercomputers



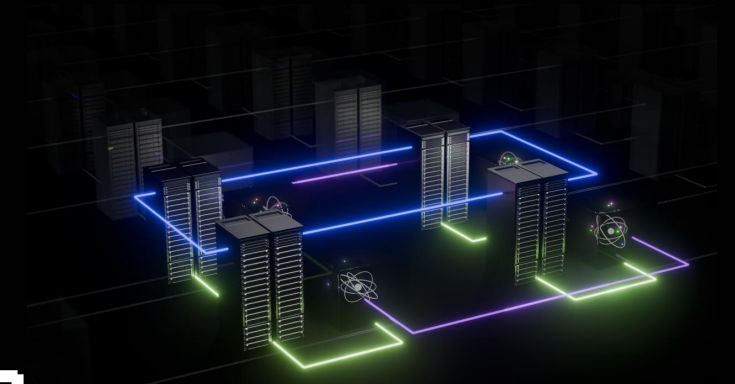
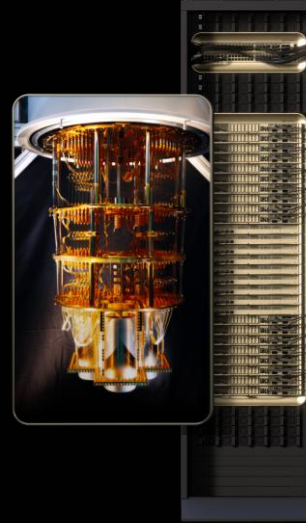
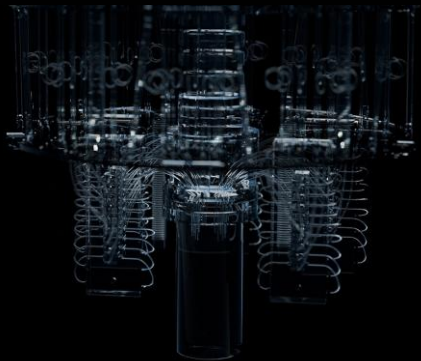
Today's
Systems



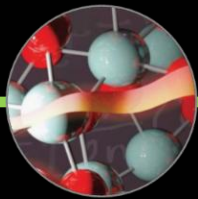
Quantum-GPU
Supercomputers

The Path from Qubits to Supercomputers

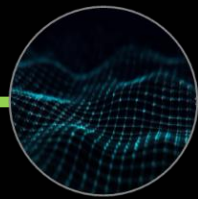
AI to the Rescue



Today's
Systems



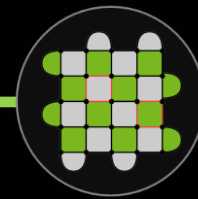
Algorithm
and Application
Development



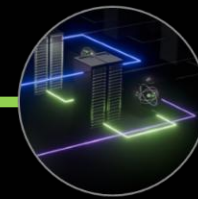
Better Hardware
Qubit EDA



Quantum Data
Generation



Real-time
Quantum Error
Correction

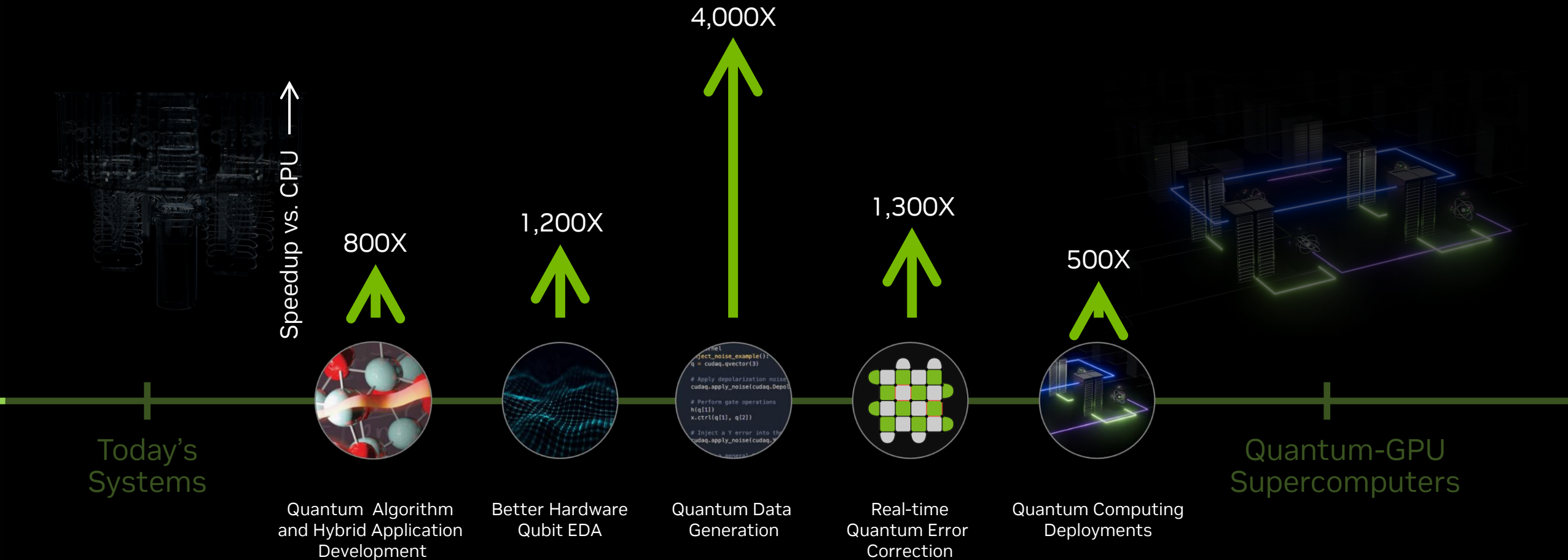


Quantum Computing
Deployments

Quantum-GPU
Supercomputers

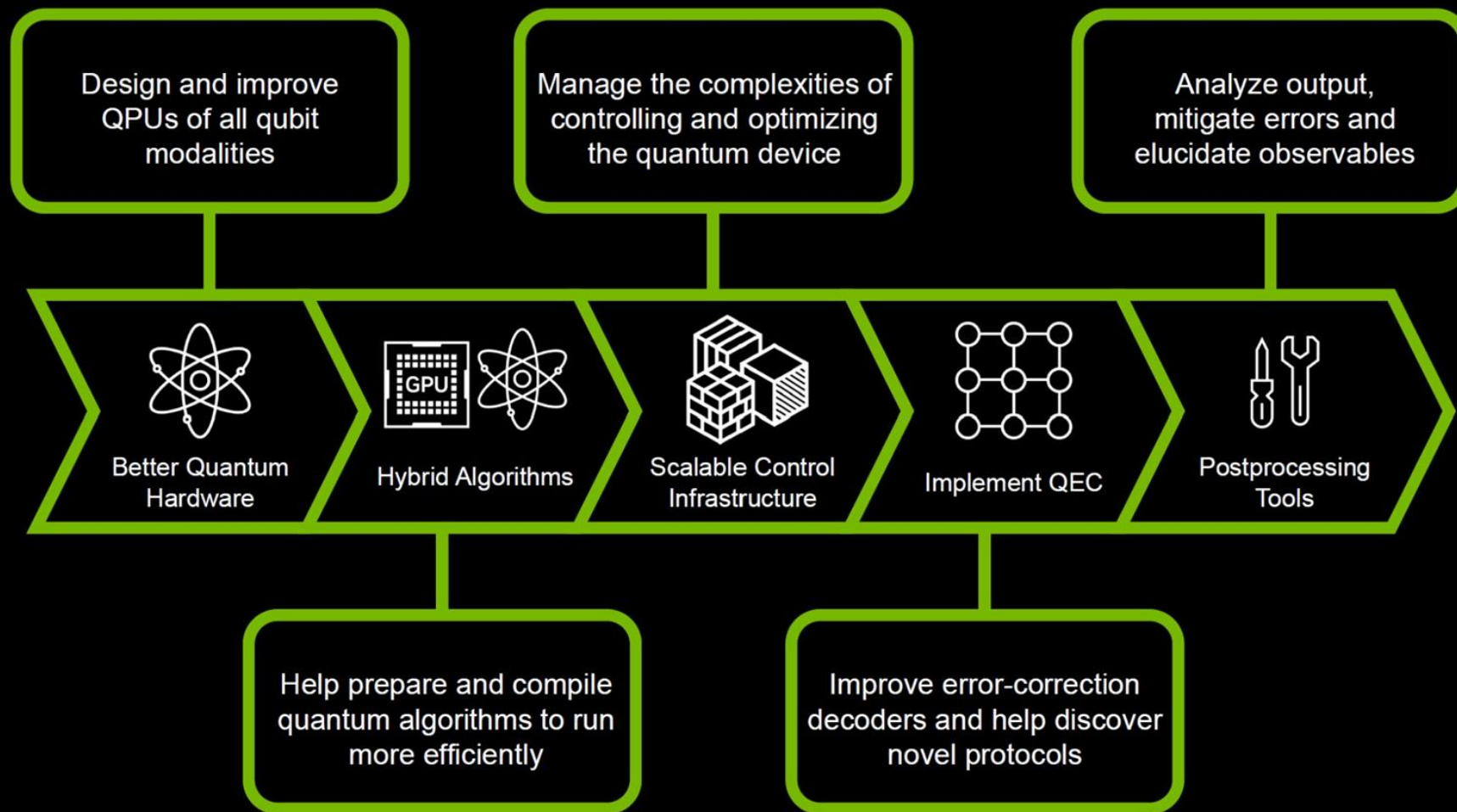
The Path from Qubits to Supercomputers

AI to the Rescue

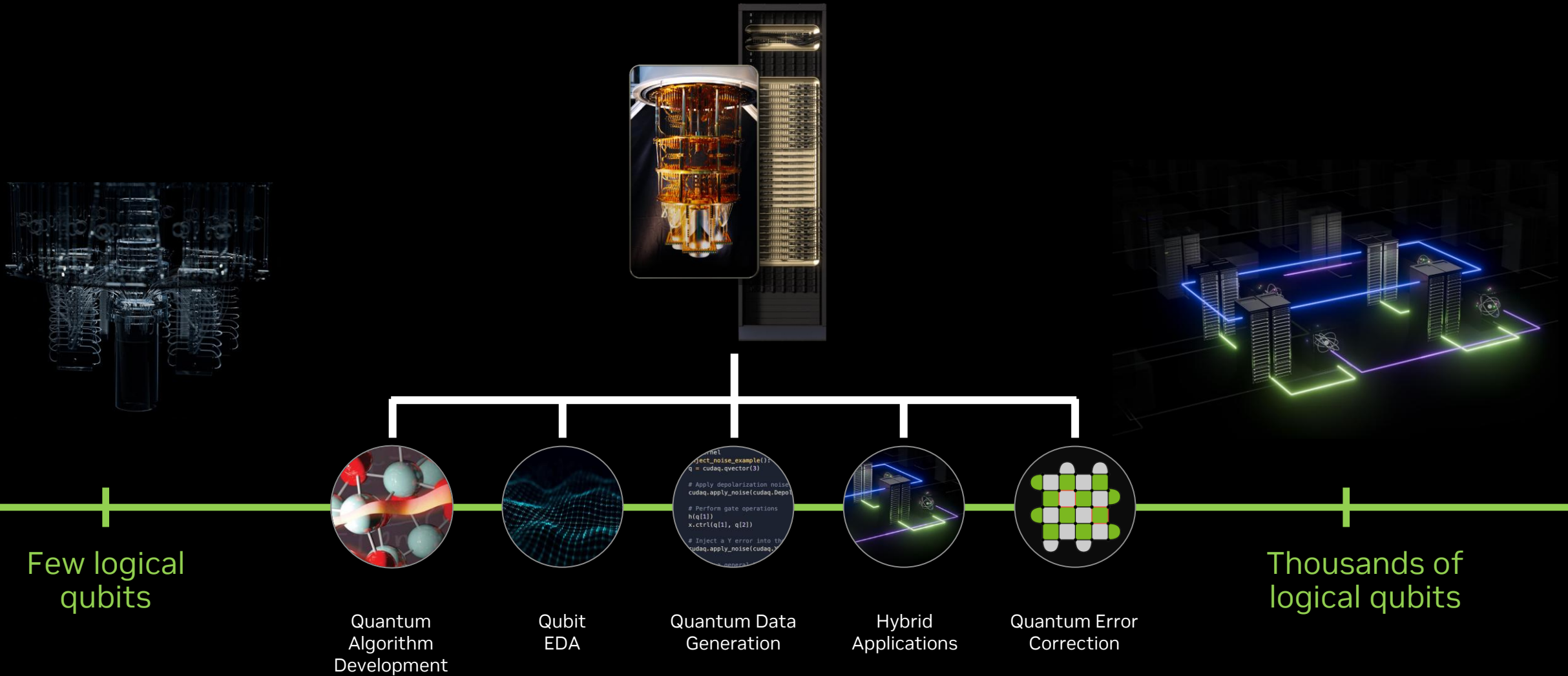


Artificial Intelligence for Quantum Computing

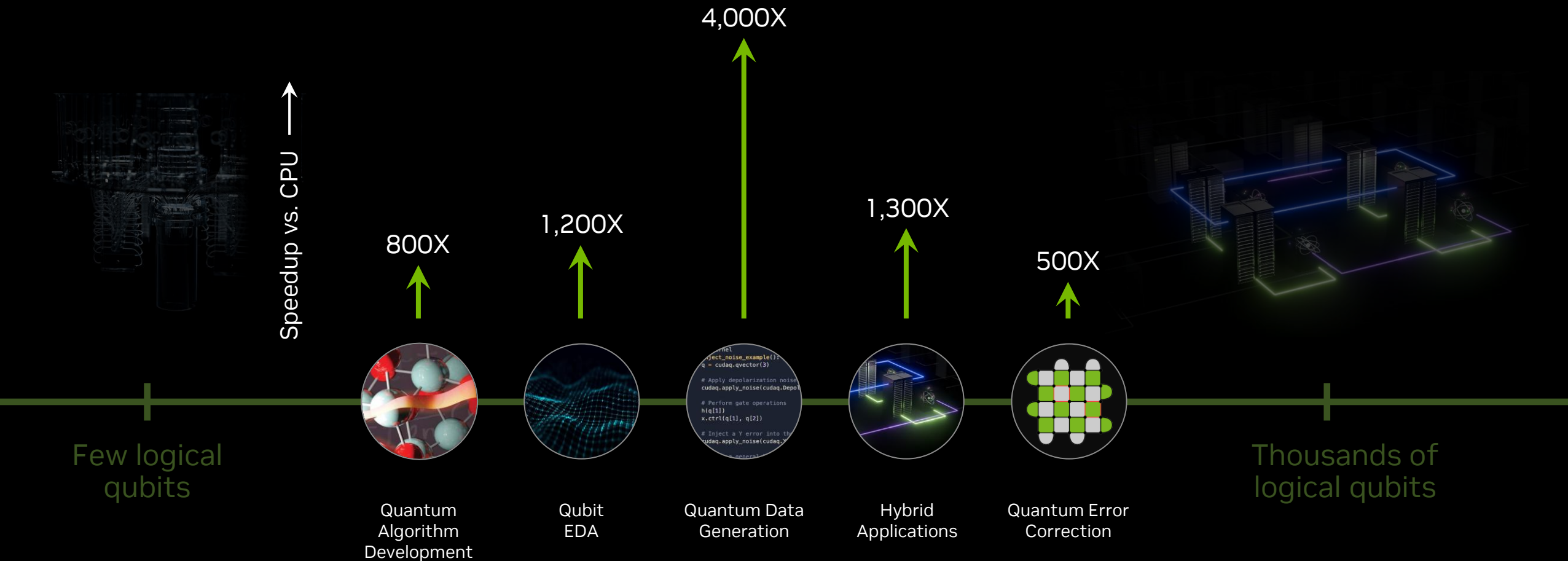
[arXiv:2411.09131](https://arxiv.org/abs/2411.09131)



The Journey to Accelerated Quantum Supercomputing



The Journey to Accelerated Quantum Supercomputing



Lab 0:
Foundations of Quantum Computing with CUDA-Q

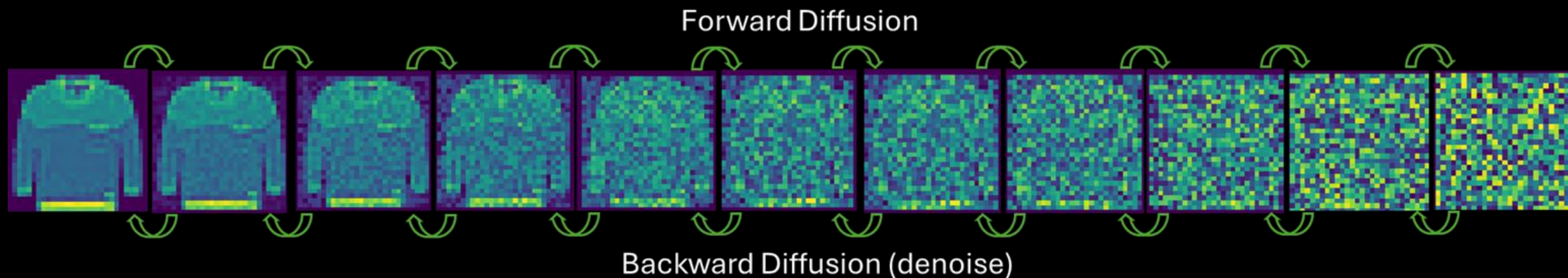


Lab 1: **Generative AI for Unitary Compilation**

Diffusion Model

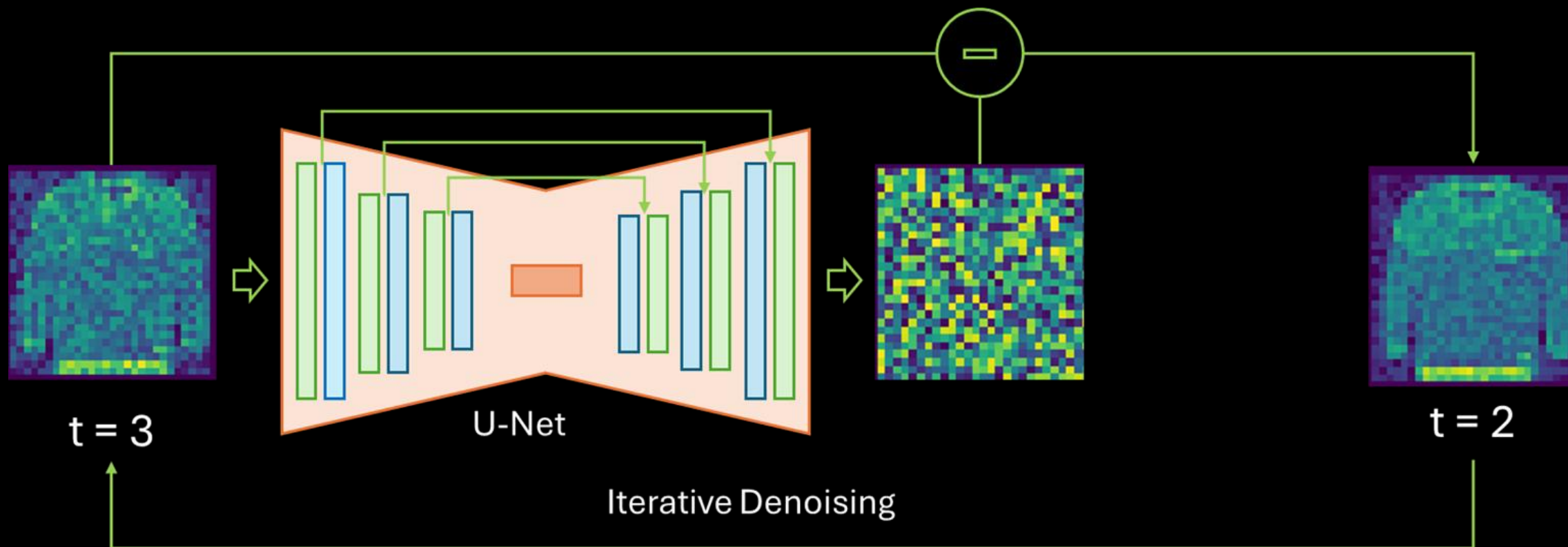
Background

- Generative AI to generate high-quality image (data)
- Inspired by physical diffusion: gradual spreading from high to low concentration
- **Forward Diffusion:** Add noise to data
- **Backward Diffusion:** Denoise to generate realistic data



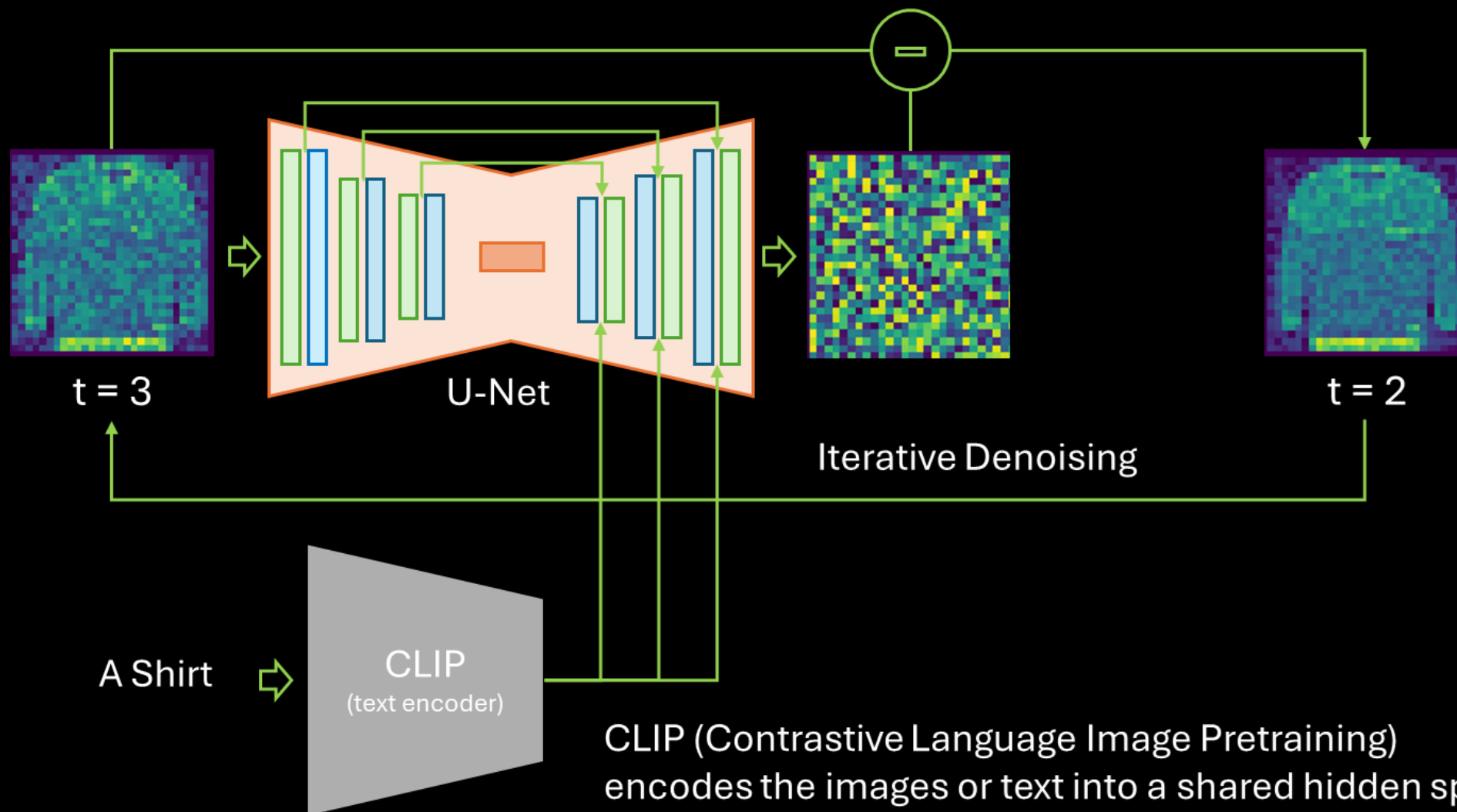
Diffusion Model

Diffusion Process



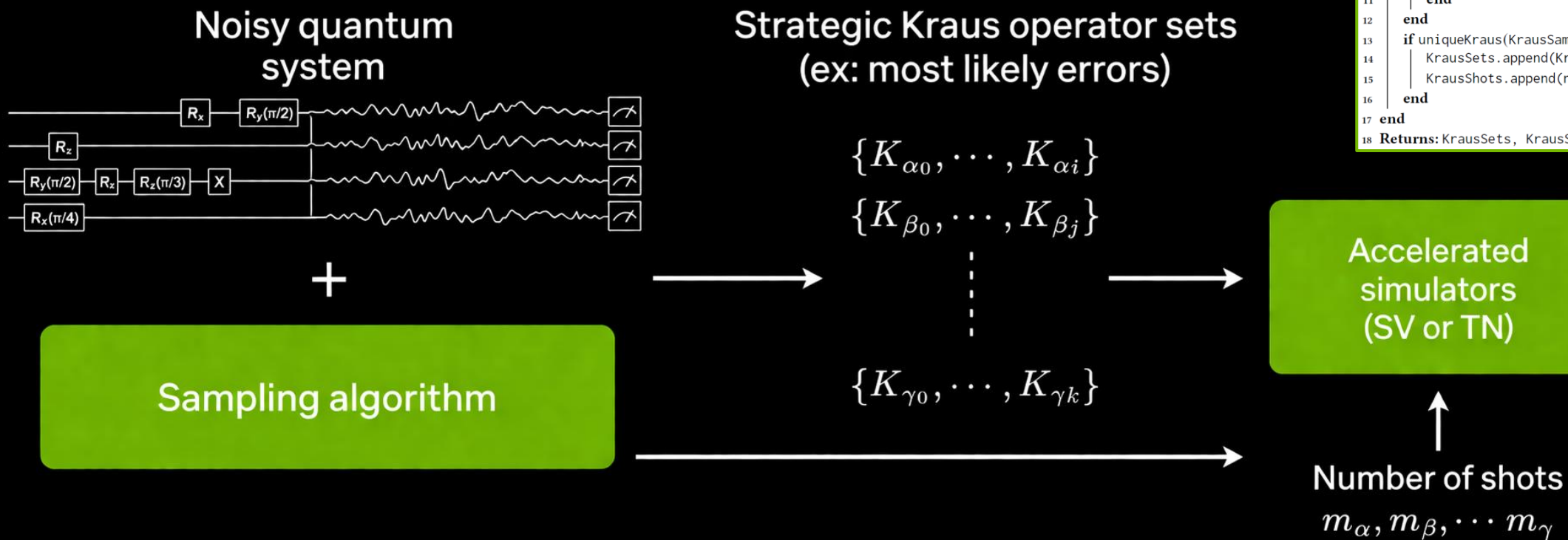
Diffusion Model

Context Controlling



Circuit Validation with Efficient Noisy Simulation

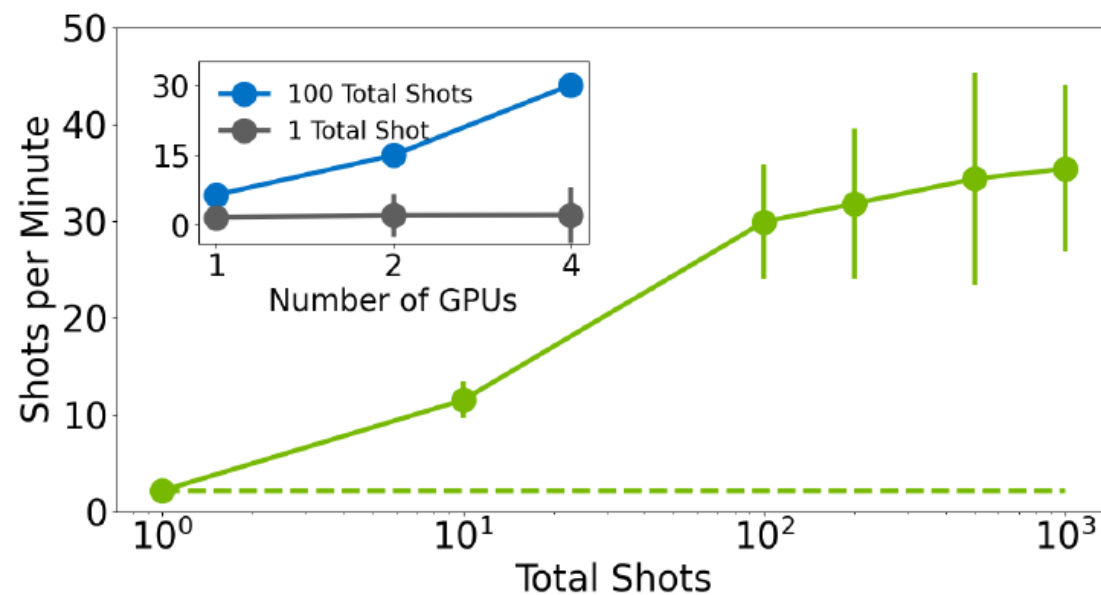
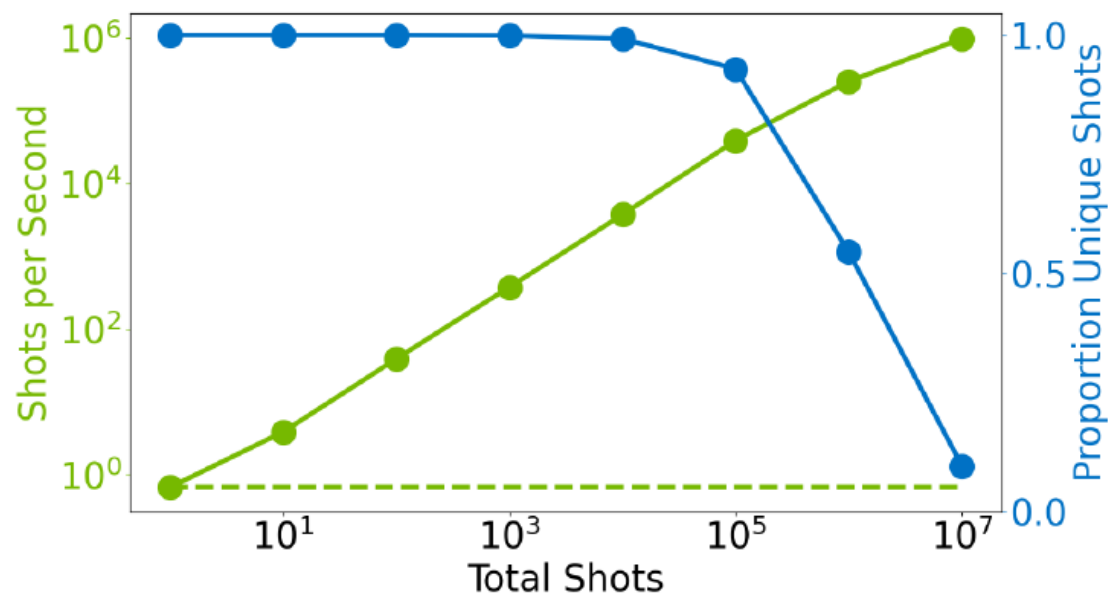
Pre-Trajectory Sampling with Batch Execution (PTSBE)



```
1 Inputs: NoisyCircuit( $\{K\}, \{p\}$ ), nsamples, nshots
2 KrausSets, KrausShots = [], []
3 for sample in range(nsamples) do
4     KrausSample = []
5     for  $K, p$  in NoisyCircuit( $\{K\}, \{p\}$ ) do
6          $r \leftarrow \text{randomUniform}(0, 1)$ ;
7         if  $r \leq p$  then
8             if compatible( $K$ , KrausSample) then
9                 KrausSample.append( $K$ );
10            end
11        end
12    end
13    if uniqueKraus(KrausSample, KrausSets) then
14        KrausSets.append(KrausSample);
15        KrausShots.append(nshots);
16    end
17 end
18 Returns: KrausSets, KrausShots
```

Circuit Validation with Efficient Noisy Simulation

Pre-Trajectory Sampling with Batch Execution (PTSBE)





Lab 2: **Transformer-Based Quantum Optimization**

Transformer Intrinsic Optimizer for QAOA

Hybrid Efficient Classical-Quantum Optimization Framework

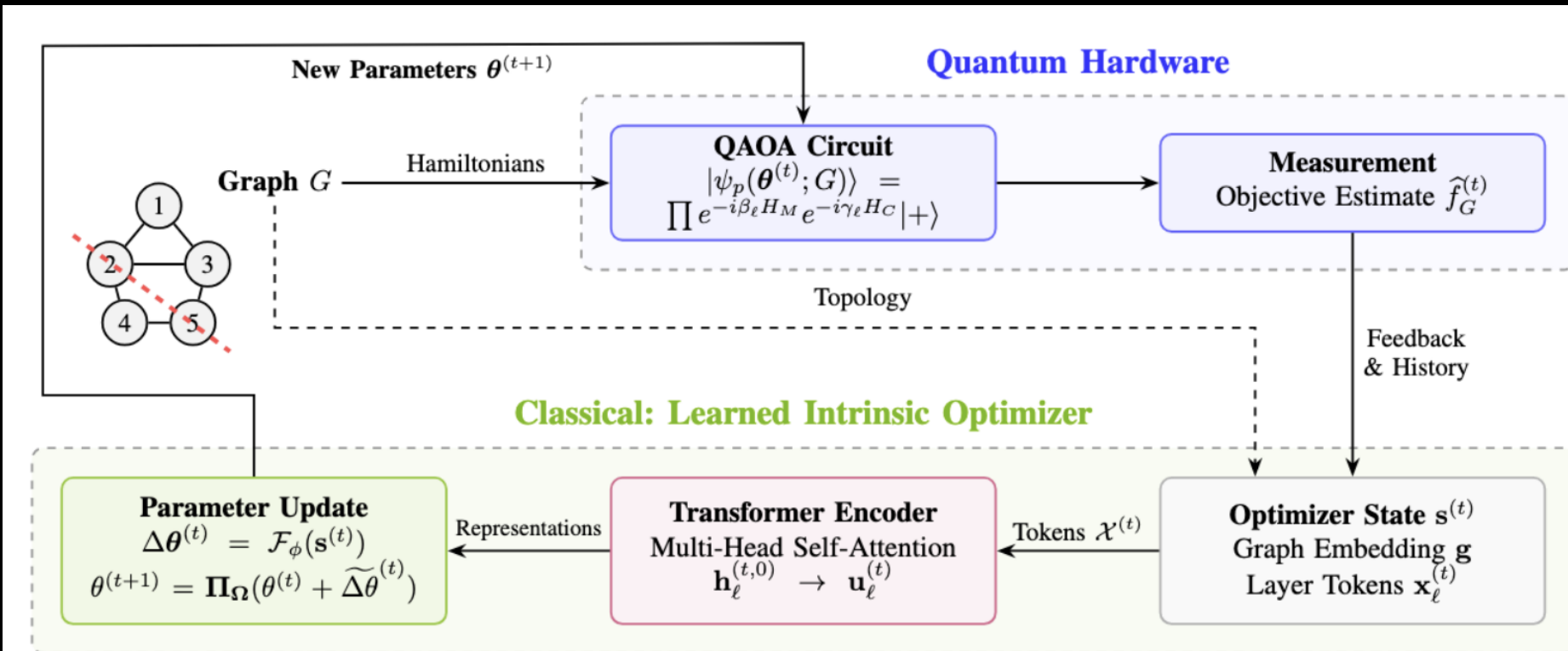


Fig. 1: Overview of the framework with the MaxCut graph shifted independently of the label.



Transformer Intrinsic Optimizer for QAOA

LEARNED UPDATE RULE

The model replaces a hand-designed classical step with a graph-conditioned, history-conditioned policy.

1) Optimizer state

$$s^{(t)} = (G, \theta^{(t)}, \Delta\theta^{(t-1)}, \hat{f}_G^{(t)}, \hat{f}_{G,\text{best}}^{(t)}, t/T)$$

State includes topology, current angles, previous step, current objective, best-so-far objective, and normalized time.

2) QAOA layer tokens

$$x_\ell^{(t)} = [\gamma_\ell^{(t)}, \beta_\ell^{(t)}, \Delta\gamma_\ell^{(t-1)}, \Delta\beta_\ell^{(t-1)}, \hat{f}_G^{(t)}, \hat{f}_{G,b}^{(t)}, t/T]^\top$$

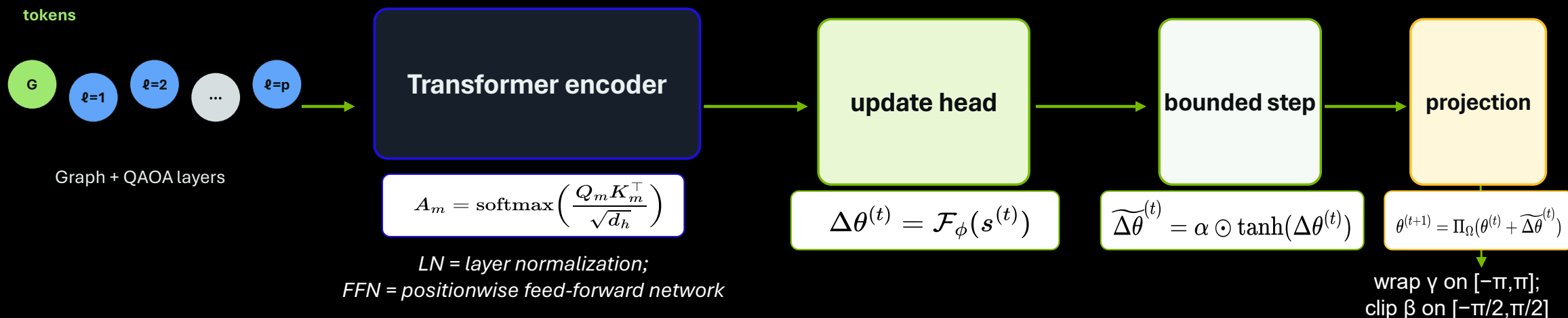
$$h_\ell^{(t,0)} = W_x x_\ell^{(t)} + e_\ell, \quad h_G^{(0)} = W_g \phi_G(G)$$

Superscript 0 denotes the input embedding before the first Transformer block.

3) Token sequence

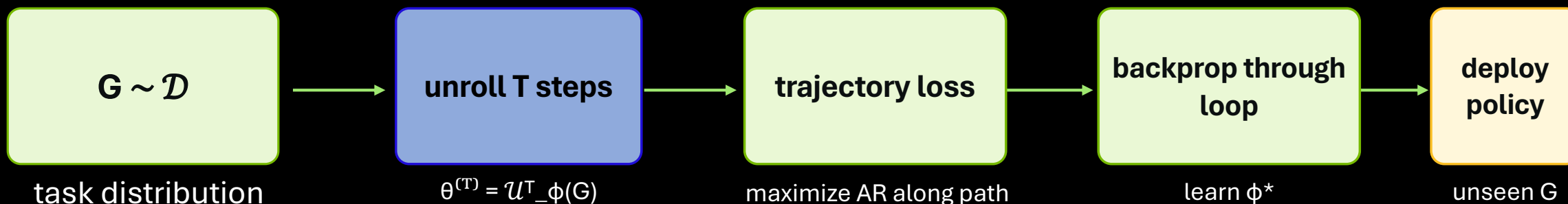
$$\mathcal{X}^{(t)} = [h_G^{(0)}, h_1^{(t,0)}, \dots, h_p^{(t,0)}]$$

Graph token attends to QAOA layer tokens; layer tokens attend to each other.



Transformer Intrinsic Optimizer for QAOA

Meta-Training and HPC-Oriented Execution



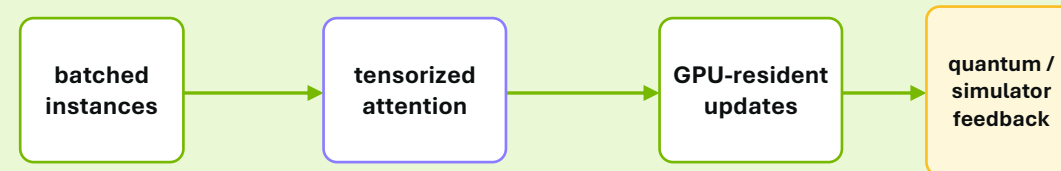
Meta-objective

$$\phi^* = \arg \min_{\phi} \mathbb{E}_{G \sim \mathcal{D}} [\mathcal{L}(G; \phi)]$$

$$\mathcal{L}_{\text{traj}}(G; \phi) = - \sum_{t=0}^T \lambda_t \frac{f_G(\theta_{\phi}^{(t)})}{C_G^*}$$

$$\mathcal{L} = \mathcal{L}_{\text{traj}} + \eta_1 \sum_t \|\widehat{\Delta \theta}^{(t)}\|_2^2 + \eta_2 \sum_t \widehat{\sigma}_G^{2(t)}$$

Accelerator-native interpretation



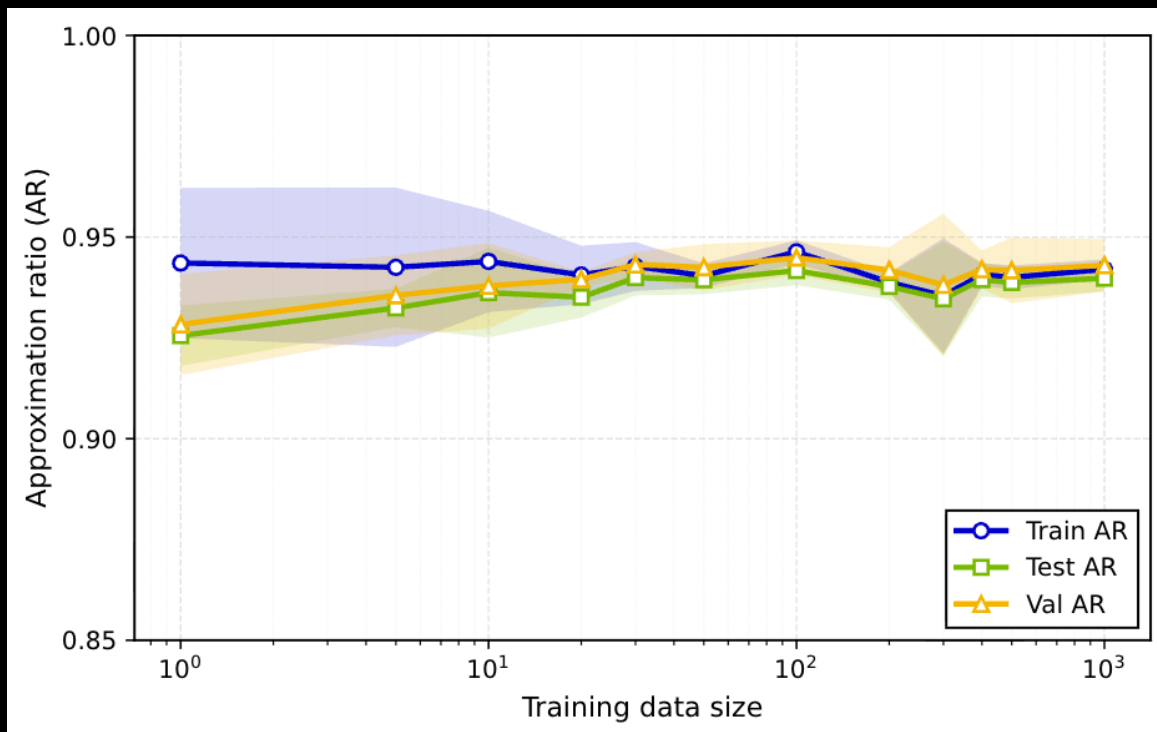
HPC takeaway: online optimization becomes a small Transformer inference workload; expensive objective calls are reserved for feedback, not blind search.

Formulation claim: learn a structured, history-dependent preconditioner $\Psi\phi(G, \theta, H_t)$ without explicit analytic gradients.

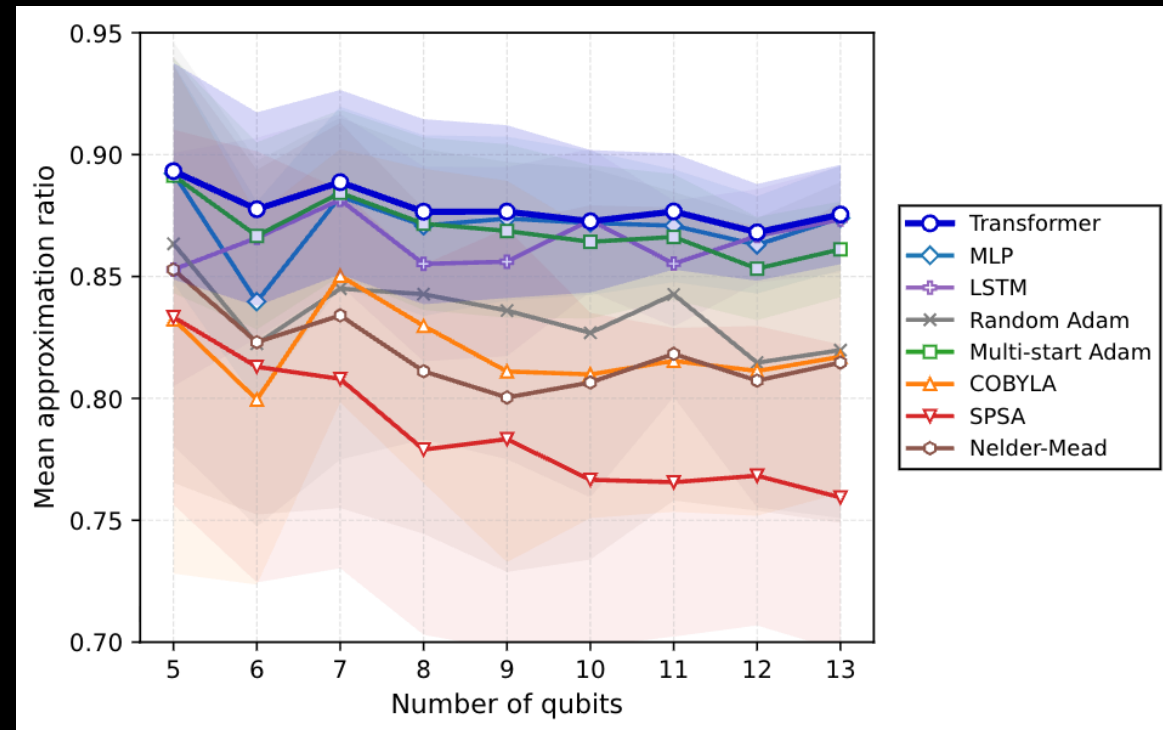
Transformer Intrinsic Optimizer for QAOA

Efficient Learning Result

Case with 6 qubit QAOA graph setting



For training the transformer optimizer for QAOA, a practical number of training graphs is sufficient.



The performance scaling is more stable than that of other optimizers.



Lab 3:

Quantum Kolmogorov-Arnold Networks (QKAN-LLM)

Kolmogorov-Arnold Network

Emergent neural network architecture

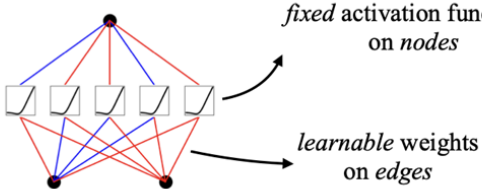
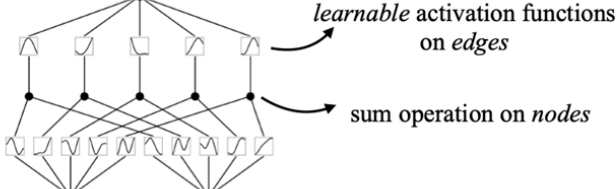
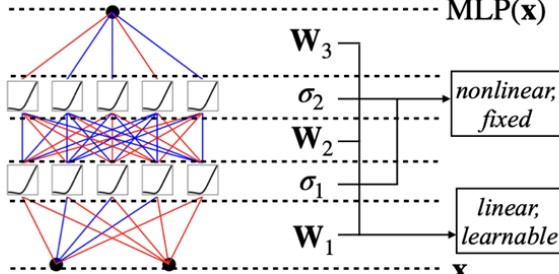
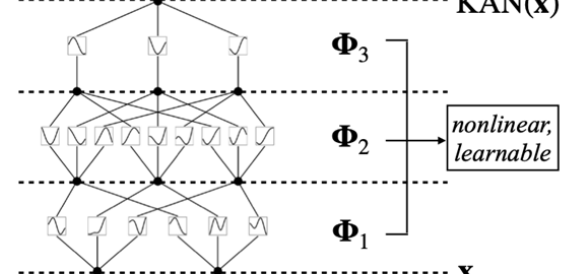
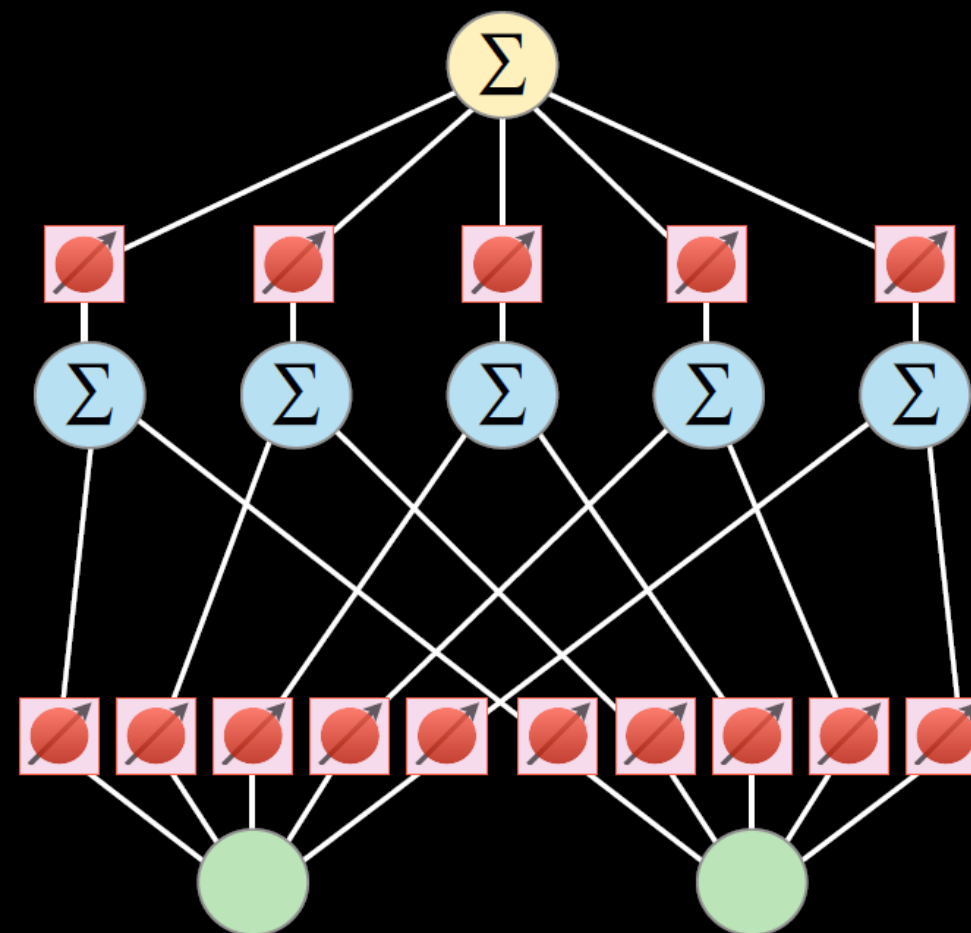
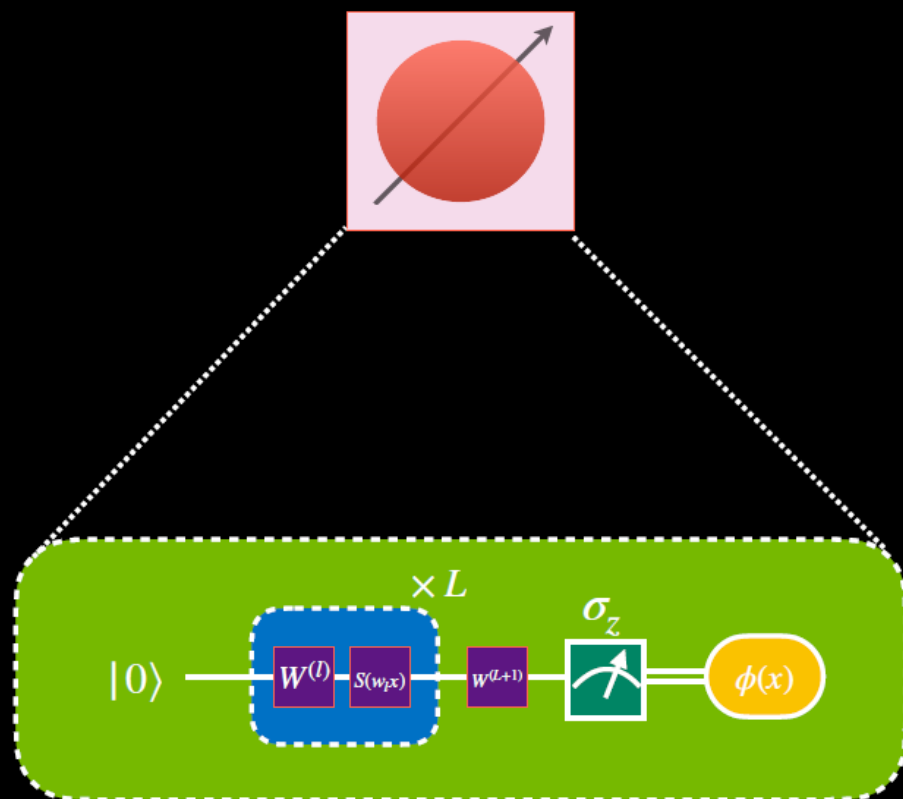
Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(\epsilon)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)	(a)  fixed activation functions on nodes learnable weights on edges	(b)  learnable activation functions on edges sum operation on nodes
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Model (Deep)	(c)  $\mathbf{W}_3$ σ_2 $\mathbf{W}_2$ σ_1 $\mathbf{W}_1$ $\mathbf{x}$ nonlinear, fixed linear, learnable	(d)  Φ_3 Φ_2 Φ_1 $\mathbf{x}$ nonlinear, learnable

Figure 0.1: Multi-Layer Perceptrons (MLPs) vs. Kolmogorov-Arnold Networks (KANs)

Quantum-inspired Kolmogorov-Arnold Network

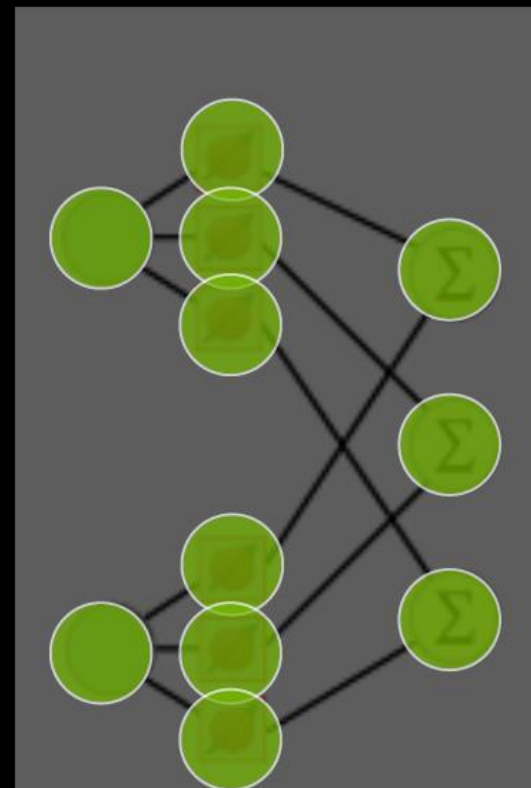
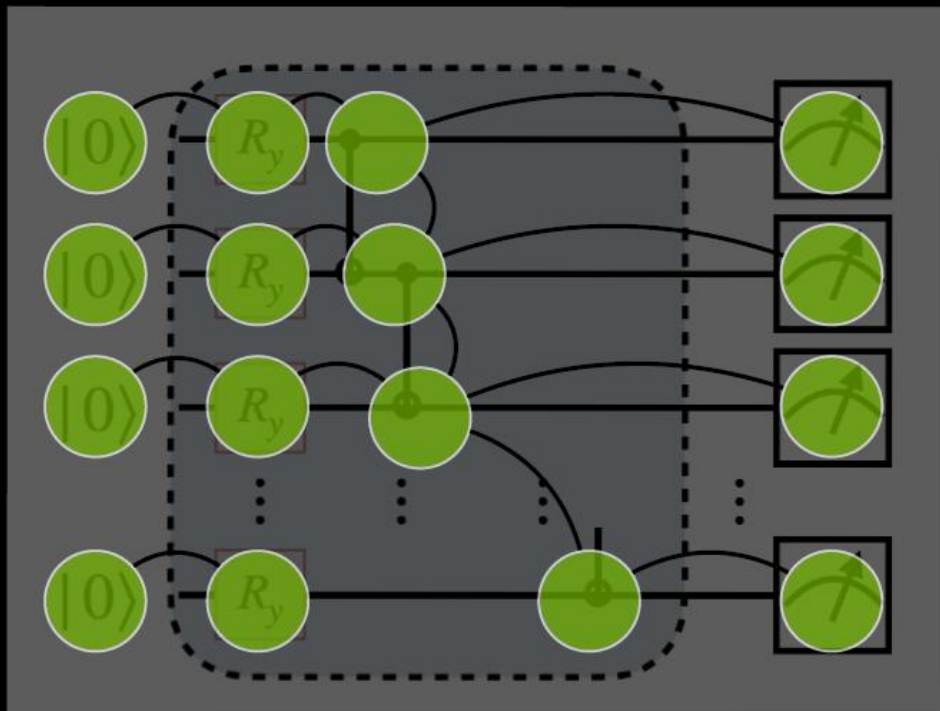
Quantum-inspired method toward parameter-efficient approach

Data Re-Uploading Activation
DARUAN



Quantum-inspired Kolmogorov-Arnold Network

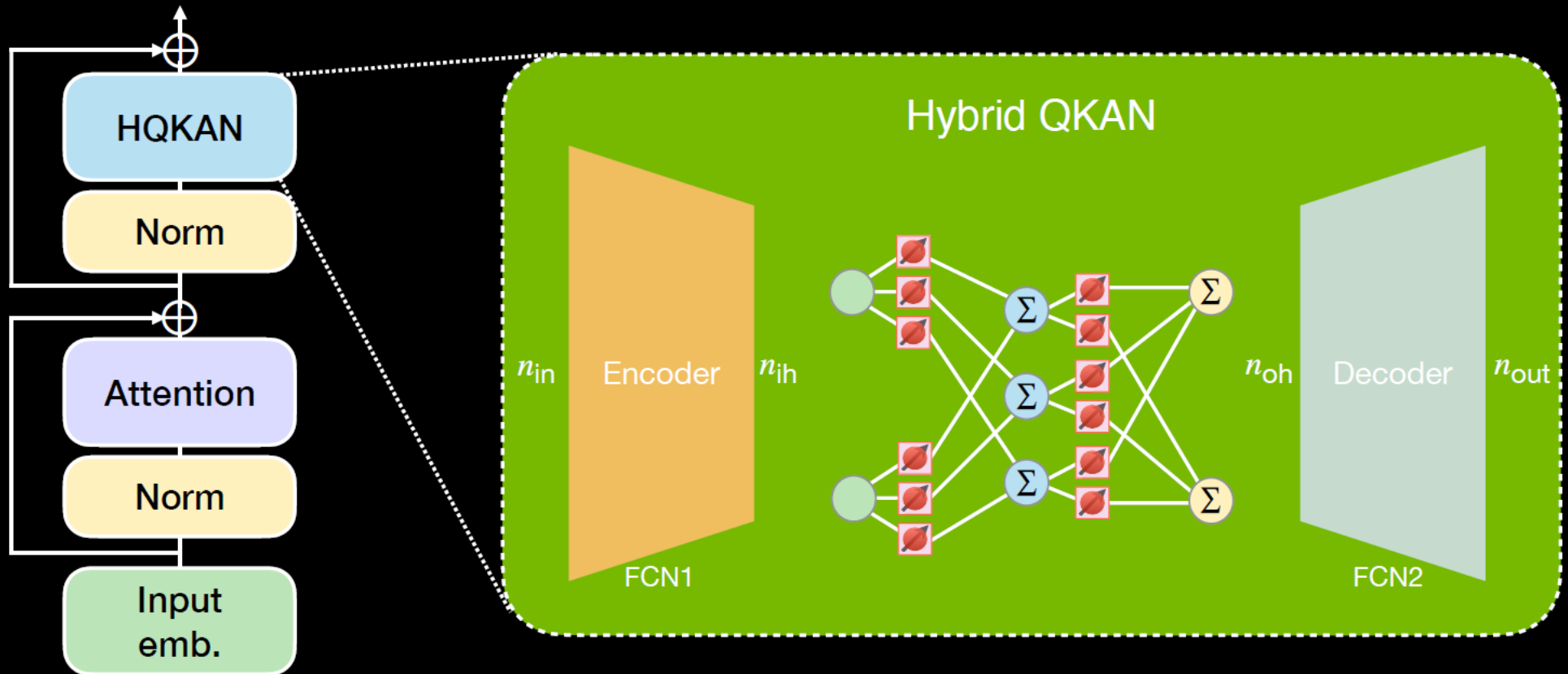
TN-accelerate QKANLayer simulation



LLM x QKAN

QKAN for quantum-enhanced language model

- HQKANsformer for parameter/computational efficient and scalable GenAI



LLM x QKAN

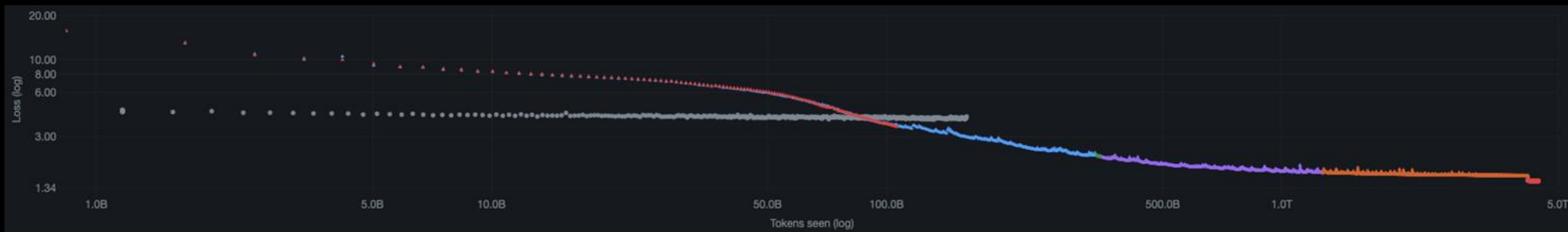
QKAN for quantum-enhanced language model (validated at ~100M-parameter scale)

Table 4: Performance comparison of GPT-2 models on the WebText dataset [53]. This table presents the runtime and memory usage for various GPT-2 configurations equipped with QKAN-based modules. The top section (above the second double midrule) corresponds to models using the **kan-gpt** framework introduced in the main text, where all linear layers are replaced by HQKAN. The models marked with an asterisk (*) are those displayed in Figure 4. Results include training speed (ms per iteration) and memory consumption. The lower section (below the second double midrule) reports large-scale results using the **KANsformer** [57] architecture with flash attention [58], which integrates HQKAN directly into the feed-forward network of transformer block instead of all linear layers. We benchmark across batch sizes and hardware configurations, including single- and multi-GPU setups with NVIDIA RTX 4090, V100S, H100, and H200 GPUs. These configurations demonstrate the scalability and practical feasibility of HQKAN-based models in both single- and distributed-training regimes.

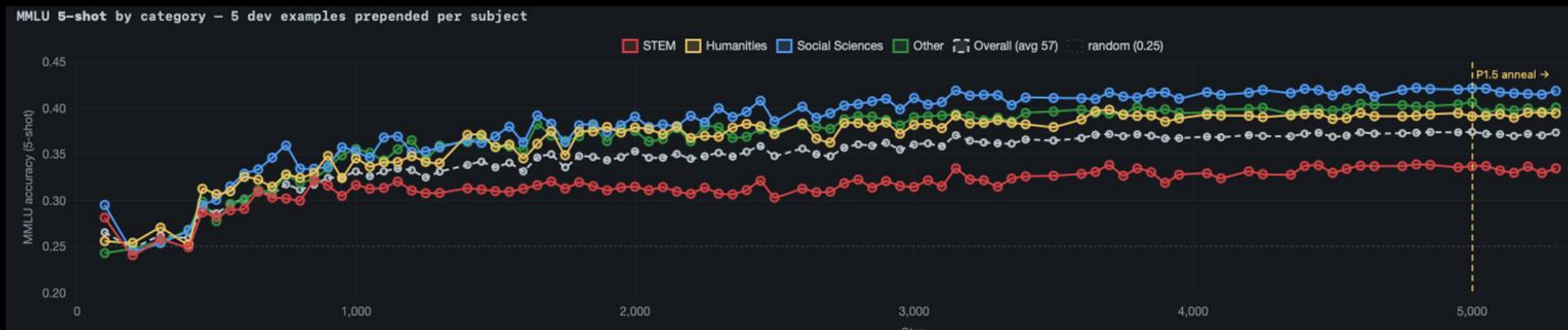
Method	# Params	Batch size	Time/iter [GPU · ms]	Device memory
MLP w/o flash atten.	124 M	*1	63 [RTX 4090 · ms]	6.5 GB
		48	4,536 [V100S · ms]	111 GB
HQKAN w/o flash atten.	40 M	*1	60 [RTX 4090 · ms]	4.1 GB
		48	4,648 [V100S · ms]	120 GB
MLP w/ flash atten.	124 M	1	40 [RTX 4090 · ms]	4.7 GB
		10	252 [RTX 4090 · ms]	17.3 GB
		48	3,884 [V100S · ms]	81.2 GB
		320	6,400 [H100 · ms]	656 GB
		800	14,784 [H200 · ms]	1,340 GB
HQKANsformer w/ flash atten.	67 M	1	39.5 [RTX 4090 · ms]	3.8 GB
		10	240 [RTX 4090 · ms]	15.6 GB
		48	3,540 [V100S · ms]	73.6 GB
		320	5,760 [H100 · ms]	592 GB
		800	13,232 [H200 · ms]	1,224 GB

LLM x QKAN

Validating scalable QLM model pre-training on 1,600 H200 GPUs HGX cluster over 2 weeks



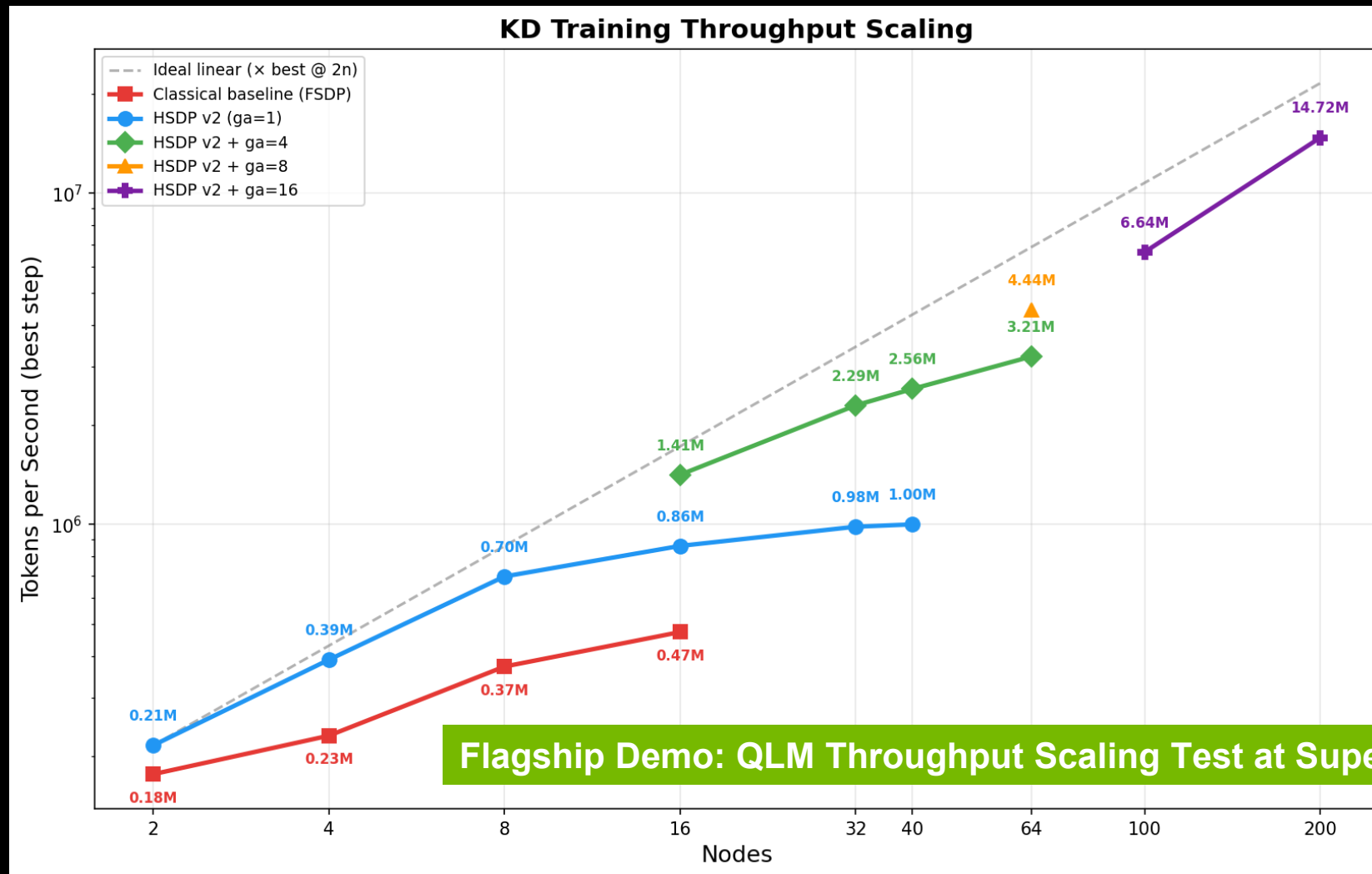
Flagship Demo: Billion-Scale Quantum-Inspired LLM Pretrained at Supercomputer Scale



(More details to come)

LLM x QKAN

Validating billion-parameter QLM throughput scaling across up to 200-node H200s



Flagship Demo: QLM Throughput Scaling Test at Supercomputer Scale

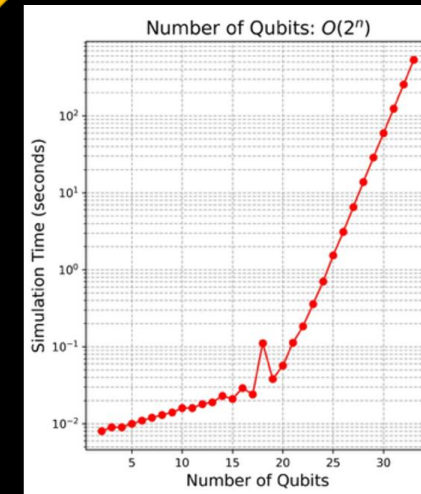
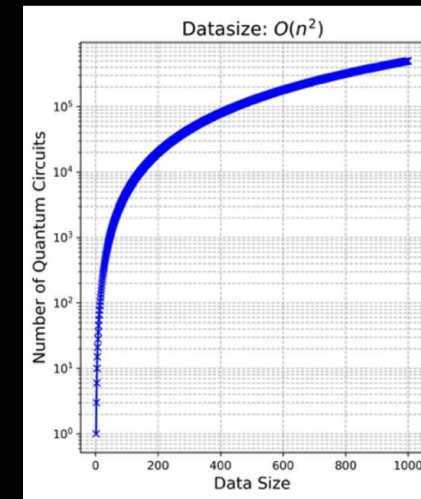
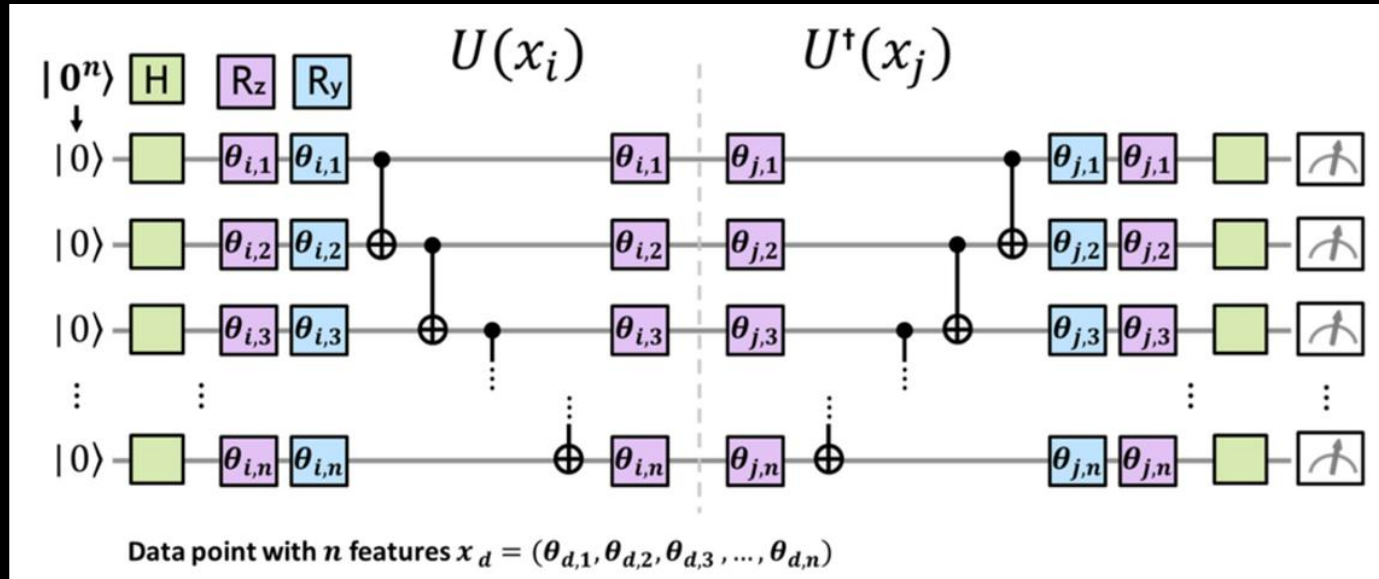
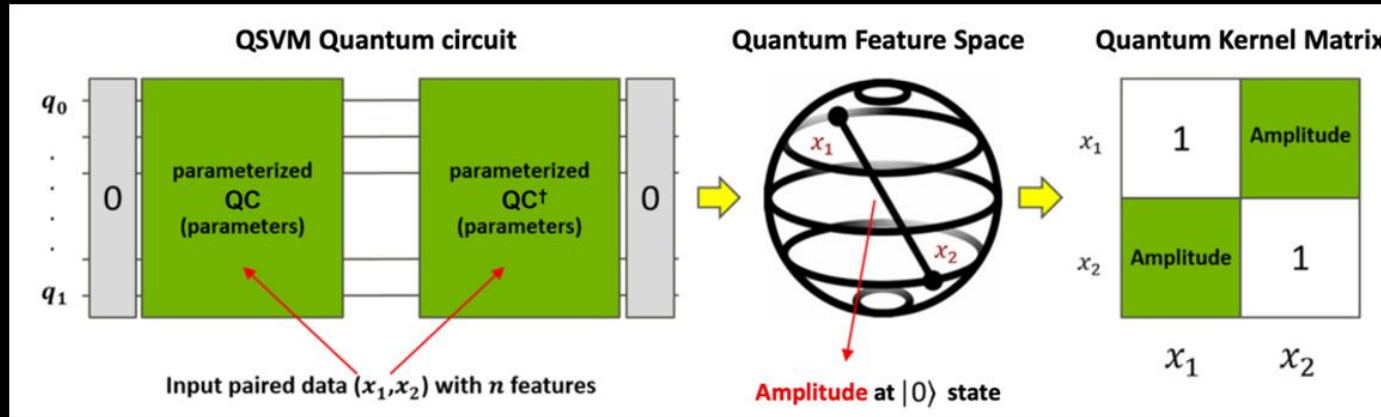
(More details to come)



Lab 4: **Quantum-Enhanced SVM (QSVM)**

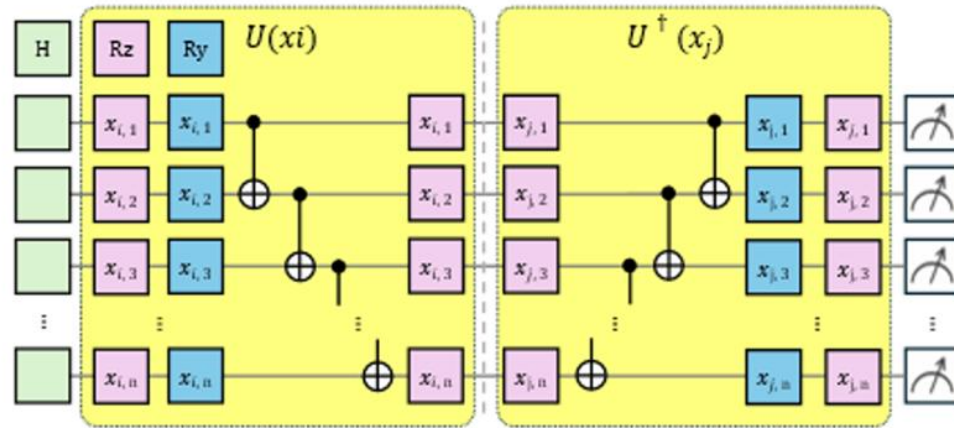
cuTN-QSVM

cuTensorNet-accelerated Quantum-enhanced Support Vector Machine

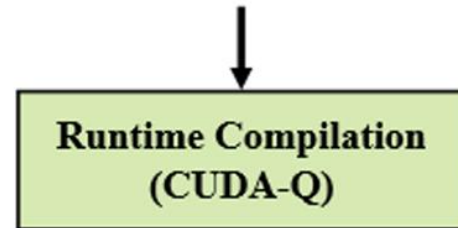


cuTN-QSVM

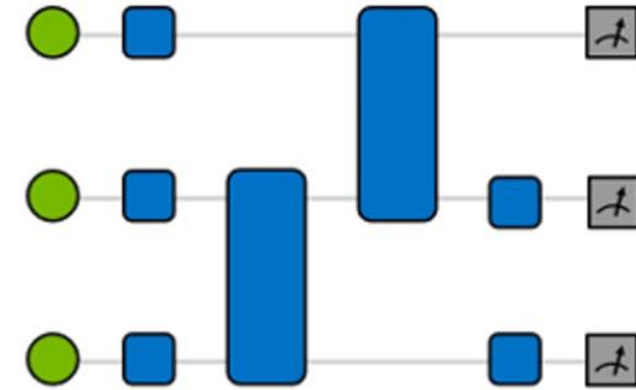
CudaqToEinsum: Efficient Tensor-Network Conversion from CUDA-Q



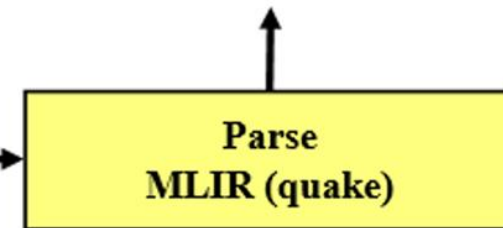
**Build Quantum Circuit From Cuda-Q
(BPS Quantum kernel)**



CudaqToEinsum

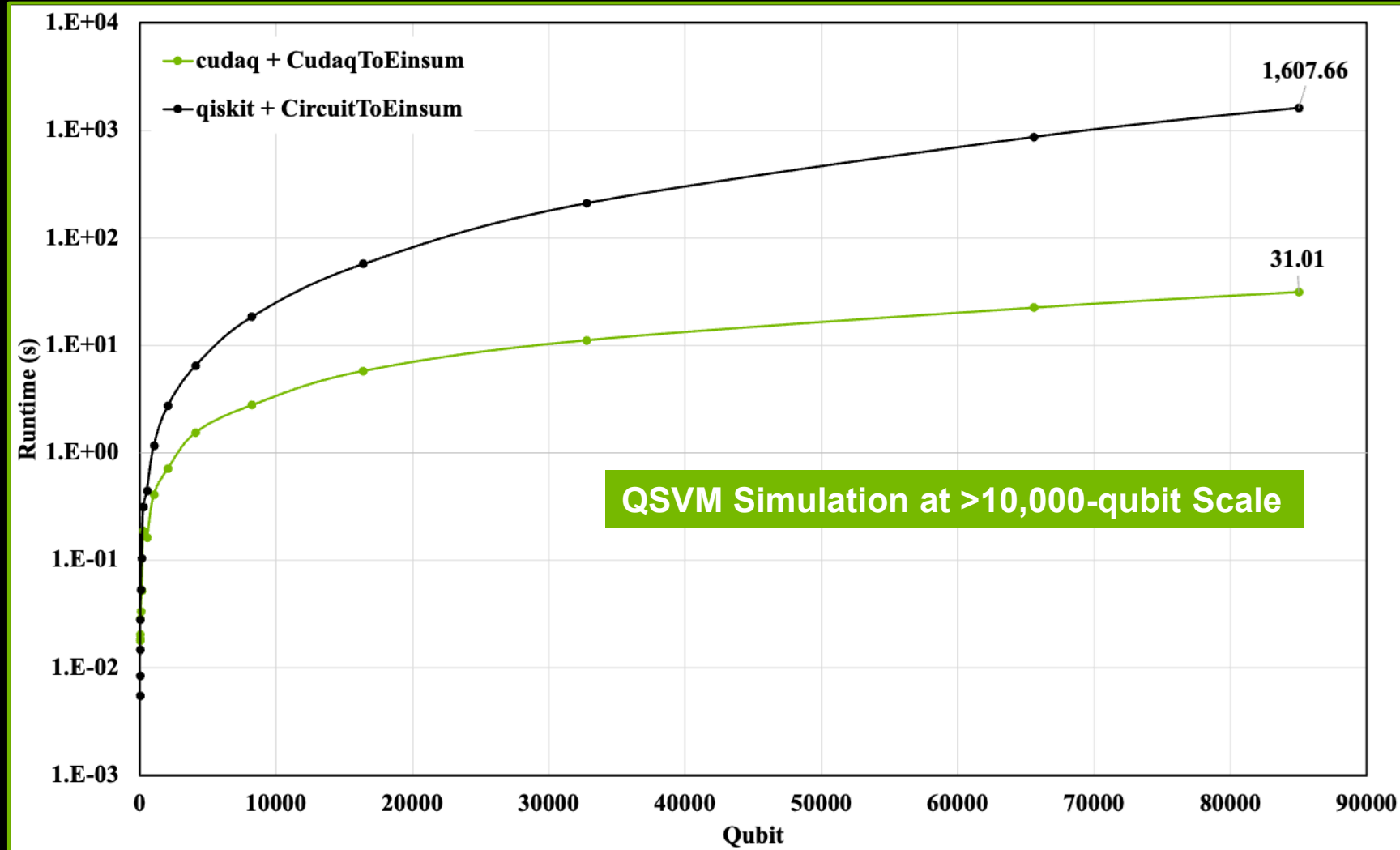


**Get Tensor Network
(Expression and Operands)**



cuTN-QSVM

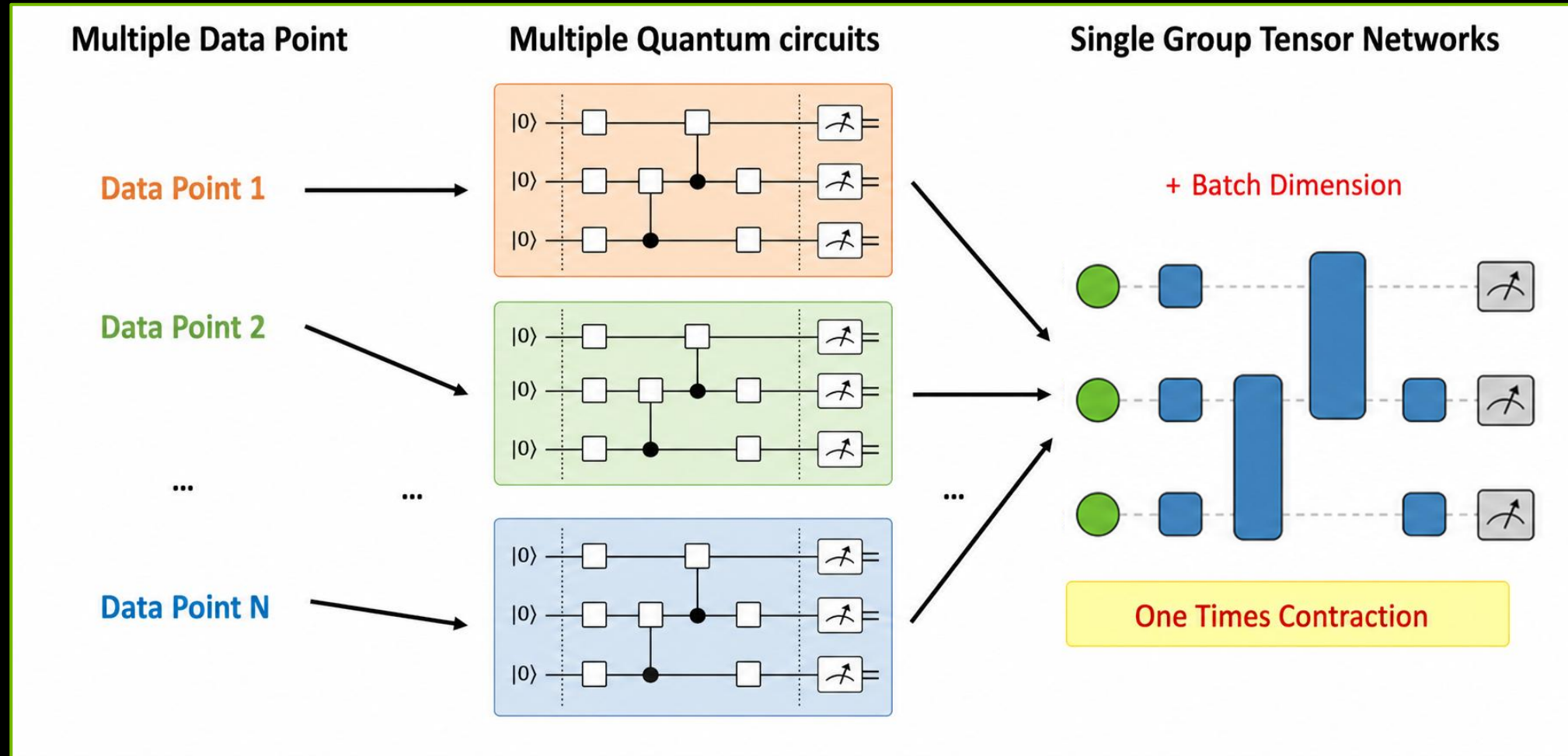
CudaqToEinsum: Efficient Tensor-Network Conversion from CUDA-Q



**x50
Speed up**

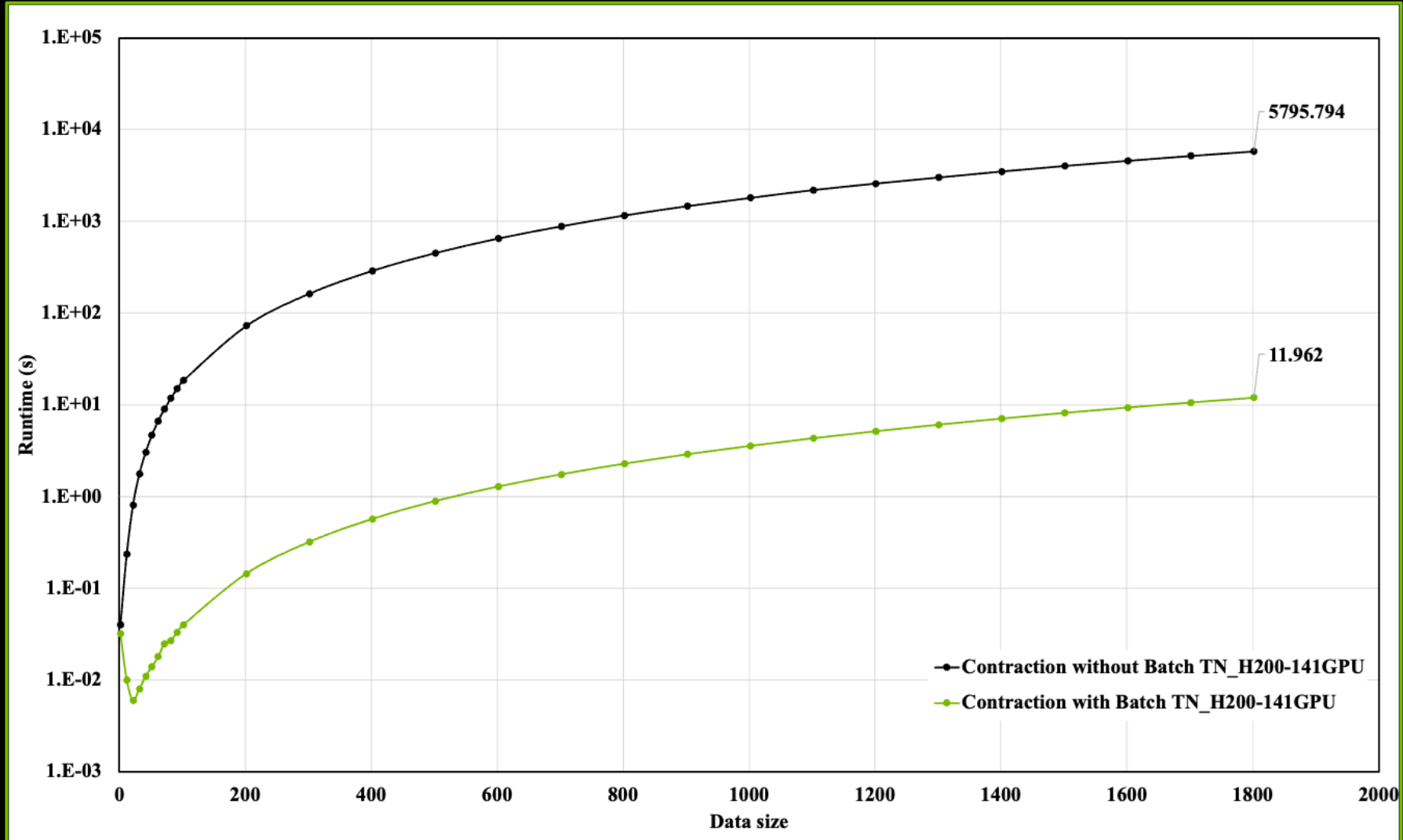
cuTN-QSVM

Efficient Batch Tensor-Network Simulation



cuTN-QSVM

Efficient Batch Tensor-Network Simulation

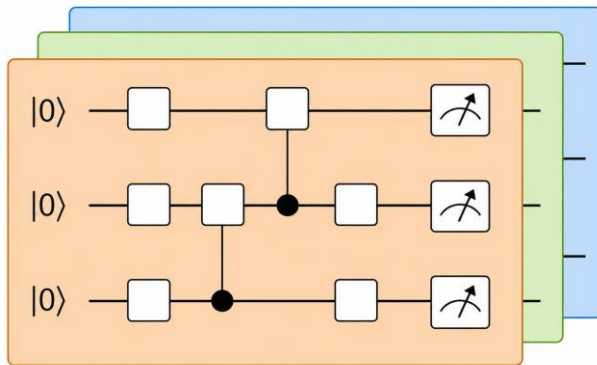


x526
Speed up

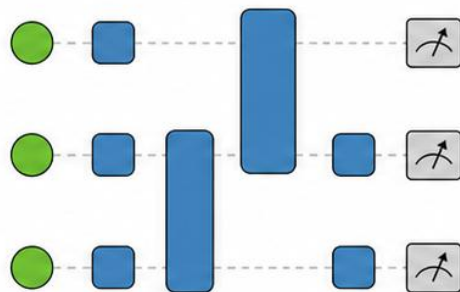
cuTN-QSVM

Multi-stream Tensor-Network Simulation Optimized by cuTENSOR

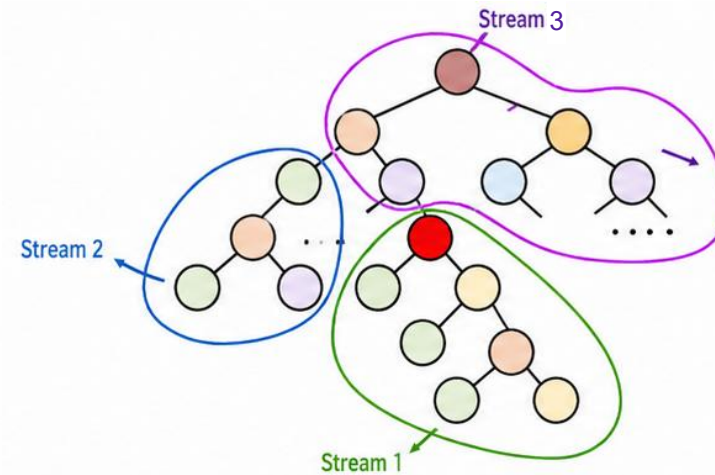
Multiple Quantum Circuits



Batch Dimension



tensor 0: abc
tensor 1: cde
tensor 2: abf
tensor 3: efg



Convert to Contraction Tree

